

Lab 8 Report: Binary Heap

Huynh Nhat Huy (523c0012)
Computer Science

November 31, 2022

1 Introduction to Binary Heap

A Heap is a special Tree-based data structure in which the tree is a complete binary tree[cite: 261]. Generally, Heaps can be categorized into two main types[cite: 262]:

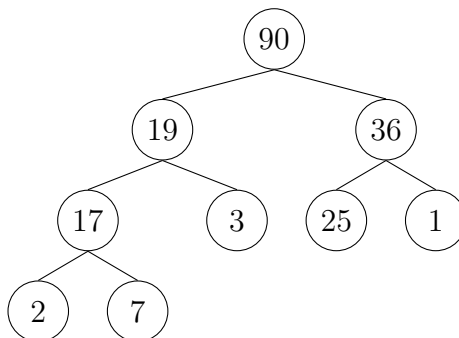
- **Max-Heap:** The key of the parent node is always greater than or equal to the keys of its children ($A[\text{parent}(i)] \geq A[i]$)[cite: 263].
- **Min-Heap:** The key of the parent node is always less than the keys of its children ($A[\text{parent}(i)] < A[i]$)[cite: 264].

For storing a complete binary tree, we can efficiently use an array[cite: 287]. If we implement a Max-heap using an array that starts at index 1[cite: 329], the relationships are defined as:

- $\text{Parent}(i) = i/2$ [cite: 328]
- $\text{Left Child}(i) = 2 * i$ [cite: 328]
- $\text{Right Child}(i) = (2 * i) + 1$ [cite: 328]

1.1 Visualizing a Max-Heap

Below is a representation of a Max-Heap structure based on the laboratory instructions.



2 Max-Heap Implementation

2.1 Initialization and Helper Methods

To construct the heap, we track the `heapSize` and the `maxSize` of the array. The root is placed at index 1, while index 0 is left unused or set to -1[cite: 350, 351, 357].

```
1 private int parent(int i) { return i / 2; }
2 private int left(int i) { return 2 * i; }
3 private int right(int i) { return (2 * i) + 1; }
4
5 private void swap(int i, int j) {
6     int temp = heap[i];
7     heap[i] = heap[j];
8     heap[j] = temp;
9 }
```

Listing 1: Helper Methods

2.2 Insertion

When a new key is inserted, the heap size is increased by 1, and the key is placed at the end of the heap[cite: 385, 386]. If the new key is larger than its parent, a `shiftUp` operation is performed to restore the Max-Heap property[cite: 387, 388].

```
1 public void insert(int key) {
2     if (heapSize == maxSize) throw new NoSuchElementException("Overflow
3     ");
4     heapSize += 1;
5     heap[heapSize] = key;
6     shiftUp(heapSize);
7 }
8 private void shiftUp(int i) {
9     while (i > 1 && heap[parent(i)] < heap[i]) {
10         swap(parent(i), i);
11         i = parent(i);
12     }
13 }
```

Listing 2: Insert and ShiftUp

2.3 Extraction (Extract Max)

Extracting the maximum element removes the root. The last element in the heap replaces the root, the heap size is decreased by 1, and a `shiftDown` operation restores the heap structure[cite: 417, 427, 428, 430, 432].

```
1 public int extractMax() {
2     if (heapSize == 0) throw new NoSuchElementException("Underflow");
3     int max = heap[1];
4     heap[1] = heap[heapSize];
5     heapSize -= 1;
6     shiftDown(1);
7     return max;
8 }
```

```

9
10 private void shiftDown(int i) {
11     while (i <= heapSize) {
12         int max_id = i;
13         if (left(i) <= heapSize && heap[max_id] < heap[left(i)]) {
14             max_id = left(i);
15         }
16         if (right(i) <= heapSize && heap[max_id] < heap[right(i)]) {
17             max_id = right(i);
18         }
19         if (max_id != i) {
20             swap(max_id, i);
21             i = max_id;
22         } else {
23             break;
24         }
25     }
26 }

```

Listing 3: Extract Max and ShiftDown

3 Heap Sort Algorithm

Heap Sort utilizes the `extractMax()` method. By continually extracting the maximum element from a built Max-Heap, we can populate an array from the end to the beginning to achieve ascending order[cite: 480, 481, 482, 483].

```

1 public static void heapSort(int[] arr) {
2     MaxHeap heap = new MaxHeap(arr.length);
3     for (int num : arr) {
4         heap.insert(num);
5     }
6     // Extracting max elements and placing them at the end of the array
7     for (int i = arr.length - 1; i >= 0; i--) {
8         arr[i] = heap.extractMax();
9     }
10 }

```

Listing 4: Heap Sort Implementation

4 Exercises Resolution

4.1 Exercise 2: Min-Heap Implementation

To convert the Max-Heap into a Min-Heap, the relational operators in the `shiftUp` and `shiftDown` methods are reversed. For instance, `heap[parent(i)] < heap[i]` becomes `heap[parent(i)] > heap[i]`.

4.2 Exercise 4: Priority Queue

A Priority Queue was implemented to manage objects representing people with names and priority numbers. Higher priority numbers indicate higher priority[cite: 506, 507]. The execution trace:

- Enqueue: (Alex, 3), (Bob, 2), (David, 6), (Susan, 1). Heap root is David.
- Dequeue: **David** is removed.
- Enqueue: (Mike, 5), (Kevin, 4). Heap root is Mike.
- Dequeue: **Mike** is removed.
- Dequeue: **Kevin** is removed.
- Enqueue: (Helen, 0), (Paul, 8), (Iris, 7). Heap root is Paul.
- Dequeue: **Paul** is removed.

The 4 persons dequeued in order are: **David, Mike, Kevin, Paul**.